Experiment No 6

Name: Devang Deokule
Roll Number: A-58
TU ID: TU4S2425009

AIM: Implementation of any one Classifier (Naïve Bayes Classifier) using JAVA.

Objectives:

After completing this experiment you will be able to:

1. List the differences among the learning types: supervised and unsupervised.
2. Apply the simple statistical learning algorithm such as Naive Bayesian Classifier to a classification task and measure the classifier's accuracy

LO's to be achieved:

LON	Lab Outcomes
LO4	Implement various data mining algorithms from scratch using languages like Python/ Java etc.

Theory:

Bayesian classifiers are statistical classifiers. They can predict class membership probabilities, such as the probability that a given tuple belongs to a particular class. Bayesian classification is based on Bayes' theorem.

Naïve Bayesian classifiers assume that the effect of an attribute value on a given class is independent of the values of the other attributes. This assumption is called *class conditional independence*. Bayesian belief networks are graphical models, which unlike naïve Bayesian classifiers, allow the representation of dependencies among subsets of attributes.

Bayes' Theorem

Let $\boldsymbol{X}$ be a data tuple. In Bayesian terms, $\boldsymbol{X}$ is considered "evidence." Let H be some hypothesis, such as that the data tuple $\boldsymbol{X}$ belongs to a specified class C. For classification problems, we want to determine $P(H|\boldsymbol{X})$, the probability that the hypothesis H holds given the "evidence" or observed data tuple $\boldsymbol{X}$. In other words, we are looking for the probability that tuple $\boldsymbol{X}$ belongs to class C, given that we know the attribute description of $\boldsymbol{X}$.

$P(\boldsymbol{X})$ is the prior probability of $\boldsymbol{X}$. Using our example, it is the probability that a person from our set of customers is 35 years old and earns $40,000.

$P(H)$ is the prior probability, or *a priori probability,* of H. For our example, this is the probability that any given customer will buy a computer, regardless of age, income, or any other information, for that matter.

$P(H|\boldsymbol{X})$ is the posterior probability, or *a posteriori probability*, of H conditioned on $\boldsymbol{X}$. For example, suppose our world of data tuples is confined to customers described by the attributes *age* and *income*, respectively, and that $\boldsymbol{X}$ is a 35-year-old customer with an income of $40,000. Suppose that H is the hypothesis that our customer will buy a computer. Then $P(H|\boldsymbol{X})$ reflects the probability that customer $\boldsymbol{X}$ will buy a computer given that we know the customer's age and income.

Similarly, $P(\boldsymbol{X}|H)$ is the posterior probability of $\boldsymbol{X}$ conditioned on H. That is, it is the probability that a customer, $\boldsymbol{X}$, is 35 years old and earns $40,000, given that we know the customer will buy a computer.

Bayes' theorem is:

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}.$$

<u>Naïve Bayesian Classification</u>

The naïve Bayesian classifier, or simple Bayesian classifier, works as follows:

1. Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n-dimensional attribute vector, $\boldsymbol{X} = (x1, x2, : : : , xn)$, depicting n measurements made on the tuple from n attributes, respectively, $A1, A2, : : : , An$.

Suppose that there are m classes, $C1, C2, : : : , Cm$. Given a tuple, $\boldsymbol{X}$, the classifier will predict that $\boldsymbol{X}$ belongs to the class having the highest posterior probability, conditioned on $\boldsymbol{X}$. That is, the naïve Bayesian classifier predicts that tuple $\boldsymbol{X}$ belongs to the class Ci if and only if

$P(C_i|\boldsymbol{X}) >$

$P(C_j|\boldsymbol{X})$ for $1 \leq j \leq m; j \neq i$:

Thus we maximize $P(Ci|\boldsymbol{X})$. The class Ci for which $P(Ci|\boldsymbol{X})$ is maximized is called the *maximum posteriori hypothesis*. By Bayes' theorem:

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}.$$

3. As $P(\boldsymbol{X})$ is constant for all classes, only $P(\boldsymbol{X}|Ci)P(Ci)$ need be maximized. If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is, $P(C1) = P(C2) = \ldots = P(Cm)$, and we would therefore maximize $P(\boldsymbol{X}|Ci)$. Otherwise, we maximize $P(\boldsymbol{X}|Ci)P(Ci)$. Note that the class prior probabilities may be estimated by $P(Ci)=|Ci,D|/|D|$,where $|Ci,D|$ is the number of training tuples of class Ci in D.

4. Given data sets with many attributes, it would be extremely computationally expensive to compute $P(\boldsymbol{X}|Ci)$. In order to reduce computation in evaluating $P(\boldsymbol{X}|Ci)$, the naive assumption of class conditional independence is made. This presumes that the values of the attributes are

conditionally independent of one another, given the class label of the tuple (i.e., that there are no dependence relationships among the attributes). Thus,

$$P(X|C_i) = \prod_{k=1}^{n} P(x_k|C_i) = P(x_1|C_i) \times P(x_2|C_i) \times \cdots \times P(x_n|C_i).$$

We can easily estimate the probabilities $P(x1|Ci)$, $P(x2|Ci)$, : : : , $P(xn|Ci)$ from the training tuples. Recall that here xk refers to the value of attribute Ak for tuple $\boldsymbol{X}$. For each attribute, we look at whether the attribute is categorical or continuous-valued. For instance, to compute $P(\boldsymbol{X}|Ci)$, we consider the following:

(a) If Ak is categorical, then $P(xk|Ci)$ is the number of tuples of class Ci in D having the value xk for Ak, divided by $|Ci,D|$, the number of tuples of class Ci in D.

(b) If Ak is continuous-valued, then the attribute is typically assumed to have a Gaussian distribution with a mean μ and standard deviation s, and perform the computations.

5. In order to predict the class label of $\boldsymbol{X}$, $P(\boldsymbol{X}|Ci)P(Ci)$ is evaluated for each class Ci. The classifier predicts that the class label of tuple $\boldsymbol{X}$ is the class Ci if and only if:

$$P(X|C_i)P(C_i) > P(X|C_j)P(C_j) \quad \text{for } 1 \le j \le m, j \ne i.$$

In other words, the predicted class label is the class Ci for which $P(\boldsymbol{X}|Ci)P(Ci)$ is the maximum.

<u>Example:</u> Predicting a class label using naïve Bayesian classification. We wish to predict the class label of a tuple using naïve Bayesian classification, given the same training data as in for decision tree induction. The training data are The data tuples are described by the attributes *age*, *income, student*, and *credit rating*. The class label attribute, *buys computer*, has two distinct values (namely, {*yes, no*}). Let $C1$ correspond to the class *buys computer = yes* and $C2$ correspond to *buys computer = no.* The tuple we wish to classify is:
$\boldsymbol{X}$ = (*age = youth, income = medium, student = yes, credit rating = fair*)

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

We need to maximize $P(\boldsymbol{X}\text{j}Ci)P(Ci)$, for $i = 1, 2$. $P(Ci)$, the prior probability of each class, can be computed based on the training tuples:

$P(buys\ computer = yes) = 9{=}14 = 0{:}643$

$P(buys\ computer = no) = 5{=}14 = 0{:}357$

To compute $P(\boldsymbol{X}|Ci)$, for $i = 1, 2$, we compute the following conditional probabilities:

$P(age = youth \mid buys\ computer = yes) = 2/9 = 0.222$

$P(age = youth \mid buys\ computer = no) = 3/5 = 0.600$

$P(income = medium \mid buys\ computer = yes) = 4/9 = 0.444$

$P(income = medium \mid buys\ computer = no) = 2/5 = 0.400$

$P(student = yes \mid buys\ computer = yes) = 6/9 = 0.667$

$P(student = yes \mid buys\ computer = no) = 1/5 = 0.200$

$P(credit\ rating = fair \mid buys\ computer = yes) = 6/9 = 0.667$

$P(credit\ rating = fair \mid buys\ computer = no) = 2/5 = 0.400$

Using the above probabilities, we obtain

$P(\boldsymbol{X}|buys\ computer = yes) = P(age = youth|\text{j}\ buys\ computer = yes)$ x

$P(income = medium \mid buys\ computer = yes)$ x $P(student = yes \mid buys\ computer = yes)$ x

$P(credit\ rating = fair|\text{j}\ buys\ computer = yes) = 0{:}222_0{:}444_0{:}667_0{:}667 = 0{:}044.$

Similarly,

$P(\boldsymbol{X}|buys\ computer = no) = 0{:}600_0{:}400_0{:}200_0{:}400 = 0{:}019.$

To find the class, Ci, that maximizes $P(\boldsymbol{X}|Ci)P(Ci)$, we compute

$P(\boldsymbol{X}|buys\ computer = yes)P(buys\ computer = yes) = 0{:}044\text{x}0{:}643 = 0{:}028$

$P(\boldsymbol{X}|buys\ computer = no)P(buys\ computer = no) = 0{:}019\text{x}0{:}357 = 0{:}007$

Therefore, the naïve Bayesian classifier predicts *buys computer = yes* for tuple ***X***.

<u>Conclusion:</u>

- Bayesian theorem is particularly suited when the dimensionality of the inputs is high. Parameter estimation for naive Bayes models uses the method of maximum likelihood. In

spite over-simplified assumptions, it often performs better in many complex real-world situations

- The accuracy is high as compared with other classification techniques.
- The Naïve Bayesian technique is computationally intensive.
- Advantage: Requires a small amount of training data to estimate the parameters.

JAVA CODE:

```
import java.util.*;
class weather{
static char outlook[]={'S','S','O','R','R','R','O','S','S','R','S','O','O','R'};
static char temperature[]={'H','H','H','M','C','C','C','M','C','M','M','M','H','M'}; static
char humidity[]={'P','P','P','P','N','N','N','P','N','N','N','P','N','P'}; static char
windy[]={'F','T','F','F','F','T','T','F','F','F','T','T','F','T'};
static char class1[]={'N','N','P','P','P','N','P','N','P','P','P','P','P','N'};
static double prob[][]=new double[4][2]; static double pp=9.0/14.0;
static double npp=5.0/14.0;
static int flag=0;
static int flag1=0;
static double play_N=1;
static double notplay_N=1;
static void cal_N(int a)
{
  if(a==1)
  {
    for(int i=0;i<4;++i)
    play_N*=prob[i][0];
    play_N*=pp;
  //System.out.println("\nValue of N of play \n"+play_N);
  }
  else
  {
    for(int i=0;i<4;++i)
    notplay_N*=prob[i][1];
    notplay_N*=npp;
  //System.out.println("\nValue of N of No play \n"+notplay_N); }
}
static double cal_play_prob(char ch)
{
        double prob=0;
        double count=0;
if(flag==0)
        {
        for(int i=0;i<14;++i)
                if(outlook[i]==ch && class1[i]=='P')
                ++count;
```

```
                prob=count/9.0;
                flag=1;
            }
    else if(flag==1)
            {
        for(int i=0;i<14;++i)
if(temperature[i]==ch && class1[i]=='P')

                ++count;
                prob=count/9.0;
                flag=2;
        }
else if(flag==2)
        {
        for(int i=0;i<14;++i)
                if(humidity[i]==ch && class1[i]=='P')
                ++count;
                prob=count/9.0;
                flag=3;
        }
Else if(flag==3)
        {
        for(int i=0;i<14;++i)
                if(windy[i]==ch && class1[i]=='P')
                ++count;
                prob=count/9.0;
        }
    return prob;
}

static double cal_noplay_prob(char ch)
{
        double prob=0;
        double count=0;
        if(flag1==0)
        {
        for(int i=0;i<14;++i)
                if(outlook[i]==ch && class1[i]=='N')
                ++count;
                prob=count/5.0;
                flag1=1;
        }
        else if(flag1==1)
        {
        for(int i=0;i<14;++i)
                if(temperature[i]==ch && class1[i]=='N')
                ++count;
```

```
                prob=count/5.0;
                flag1=2;
        }
        else if(flag1==2)
                {
                for(int i=0;i<14;++i)
                        if(humidity[i]==ch && class1[i]=='N')
                        ++count;
                        prob=count/5.0;
                        flag1=3;
}

        Else if(flag1==3)
                {
                for(int i=0;i<14;++i)
                        if(windy[i]==ch && class1[i]=='N')
                        ++count;
                        prob=count/5.0;
                }
        return prob;
  }
public static void main(String args[])
{
        Scanner scr=new Scanner(System.in);
        System.out.println("Table\n");
        System.out.println("Outlook\t   Temperature\t    Humidity\t      Windy     \tClass");
        for(int i=0;i<14;++i)
        {
        System.out.print(outlook[i]+"\t\t"+temperature[i]+"\t\t"+humidity[i]+"\t\t"+windy[i]+"\t\t
        "+
              class1[i]);
        System.out.println();
       }
System.out.println("Menu:\nOutlook: Sunny=S Overcast=O Rain=R\tTemperature: Hot=H
Mild=M Cool=C\n");
System.out.println("Humidity: Peak=P Normal=N\t\tWindy: True=T False=F\n\nYour input
should
belong to one of these classes.\n");
System.out.println("class1: Play=P   class2:Not Play=NP");
System.out.println("\nEnter your input: example. t={rain,hot,peak,false} input will be R,H,P,F");
String s=scr.nextLine();
char ch;
int count=0;
for(int i=0;i<8;i+=2)
{
        ch=s.charAt(i);
        prob[count][0]=cal_play_prob(ch);
```

```
        prob[count][1]=cal_noplay_prob(ch);
        ++count;
}
cal_N(1);
cal_N(2);
double pt=play_N+notplay_N;
double prob_of_play=0;
double prob_of_noplay=0;
prob_of_play=play_N/pt;
prob_of_noplay=notplay_N/pt;
System.out.println("\nProbability of play "+prob_of_play);
System.out.println("\nProbability of NO play "+prob_of_noplay );

if(prob_of_play>prob_of_noplay)
     System.out.println("\nThe new tuple classified under \"PLAY\" category.Hence there will
                                be play!!!");
else

    System.out.println("\nThe new tuple classified under \"NO PLAY\" category.Hence there
                              will be NO play.");
  }
}
```

OUTPUT:

```
C:\Users\VIKRANT111\Desktop>javac weather.java

C:\Users\VIKRANT111\Desktop>java weather
Table

Outlook    Temperature      Humidity          Windy             Class
S               H               P               F               N
S               H               P               T               N
O               H               P               F               P
R               M               P               F               P
R               C               N               F               P
R               C               N               T               N
O               C               N               T               P
S               M               P               F               N
S               C               N               F               P
R               M               N               F               P
S               M               N               T               P
O               M               P               T               P
O               H               N               F               P
R               M               P               T               N
Menu:
Outlook: Sunny=S Overcast=O Rain=R      Temperature: Hot=H Mild=M Cool=C

Humidity: Peak=P Normal=N               Windy: True=T False=F

Your input should belong to one of these classes.

class1: Play=P   class2:Not Play=NP

Enter your input: example. t=(rain,hot,peak,false) input will be R,H,P,F
O,C,N,F

Probability of play 1.0

Probability of NO play 0.0

The new tuple classified under "PLAY" category.Hence there will be play!!!
```

Post Lab Exercise:

Qu-1: Why is *naive Bayesian classification* called "naive"? Briefly outline the major ideas of naive Bayesian classification.

Answer: Naïve Bayesian classification is called naive because it assumes class conditional independence. That is, the effect of an attribute value on a given class is independent of the values of the other attributes. This assumption is made to reduce computational costs, and hence is considered "naive". For rest answer pls refer Han kamber.